

3510. Minimum Pair Removal to Sort Array II

Solved 

Hard

 Topics

 Companies

 Hint

Given an array `nums`, you can perform the following operation any number of times:

- Select the **adjacent** pair with the **minimum** sum in `nums`. If multiple such pairs exist, choose the leftmost one.
- Replace the pair with their sum.

Return the **minimum number of operations** needed to make the array **non-decreasing**.

An array is said to be **non-decreasing** if each element is greater than or equal to its previous element (if it exists).

Example 1:

Input: `nums = [5,2,3,1]`

Output: 2

Explanation:

- The pair `(3, 1)` has the minimum sum of 4. After replacement, `nums = [5,2,4]`.
- The pair `(2, 4)` has the minimum sum of 6. After replacement, `nums = [5,6]`.

The array `nums` became non-decreasing in two operations.

Example 2:

Input: `nums = [1,2,2]`

Output: 0

Explanation:

The array `nums` is already sorted.

Constraints:

- `1 <= nums.length <= 105`
- `-109 <= nums[i] <= 109`

Python:

import heapq

class Solution:

def minimumPairRemoval(self, nums: List[int]) -> int:

n = len(nums)

pq = []

decrease = 0

lo = list(range(n))

hi = list(range(n))

for i, (x, y) in enumerate(pairwise(nums)):

if x > y:

decrease += 1

heappush(pq, (x+y, i, i+1, x, y))

ans = 0

while decrease:

v, i, j, x, y = heappop(pq)

if x == nums[i] and y == nums[j]:

if nums[i] > nums[j]: decrease -= 1

hi[i] = hi[j]

lo[hi[j]] = i

if i:

pre = lo[i-1]

if nums[pre] > nums[i] and nums[pre] <= v:

decrease -= 1

if nums[pre] <= nums[i] and nums[pre] > v:

decrease += 1

heappush(pq, (nums[pre] + v, pre, i, nums[pre], v))

if hi[j] + 1 < n:

nxt = hi[j] + 1

if nums[j] <= nums[nxt] and v > nums[nxt]:

decrease += 1

if nums[j] > nums[nxt] and nums[nxt] >= v:

decrease -= 1

heappush(pq, (v+nums[nxt], i, nxt, v, nums[nxt]))

```

        nums[i] = v
        nums[j] = None
        ans += 1
    return ans

```

JavaScript:

```

function minimumPairRemoval(nums) {
    const n = nums.length;
    if (n <= 1) return 0;

    const a = nums.slice();
    const nxt = new Array(n).fill(0);
    const pre = new Array(n).fill(0);
    const alive = new Array(n).fill(true);

    for (let i = 0; i < n; i++) {
        nxt[i] = i + 1;
        pre[i] = i - 1;
    }

    // min-heap for pairs [sum, idx]
    const heap = [];
    const push = (sum, idx) => {
        heap.push([sum, idx]);
        let i = heap.length - 1;
        while (i > 0) {
            const p = (i - 1) >> 1;
            if (heap[p][0] > heap[i][0] || (heap[p][0] === heap[i][0] && heap[p][1] > heap[i][1])) {
                [heap[p], heap[i]] = [heap[i], heap[p]];
                i = p;
            } else break;
        }
    };

    const pop = () => {
        if (!heap.length) return null;
        const top = heap[0];
        const last = heap.pop();
        if (heap.length) {
            heap[0] = last;
            let i = 0;
            while (true) {
                let l = i * 2 + 1, r = i * 2 + 2, smallest = i;

```

```

        if (l < heap.length && (heap[l][0] < heap[smallest][0] || (heap[l][0] === heap[smallest][0] &&
heap[l][1] < heap[smallest][1]))) smallest = l;
        if (r < heap.length && (heap[r][0] < heap[smallest][0] || (heap[r][0] === heap[smallest][0] &&
heap[r][1] < heap[smallest][1]))) smallest = r;
        if (smallest === i) break;
        [heap[i], heap[smallest]] = [heap[smallest], heap[i]];
        i = smallest;
    }
}
return top;
};

```

```

// initial heap of adjacent pairs
for (let i = 0; i < n - 1; i++) push(a[i] + a[i + 1], i);

```

```

// count inversions (places that violate non-decreasing)
let inv = 0;
for (let i = 0; i < n - 1; i++) if (a[i] > a[i + 1]) inv++;

```

```

let ans = 0;
while (inv > 0 && heap.length) {
    const top = pop();
    if (!top) break;
    const [sum, i] = top;
    if (!alive[i]) continue;
    const j = nxt[i];
    if (j >= n || j < 0 || !alive[j]) continue;
    if (a[i] + a[j] !== sum) {
        // values changed, stale entry
        continue;
    }
}

```

```

// merge i and j into i
const p = pre[i], q = nxt[j];

```

```

// adjust inversion count: remove relations that will disappear
if (p >= 0 && alive[p]) {
    if (a[p] > a[i]) inv--;
}
if (alive[i] && alive[j]) {
    if (a[i] > a[j]) inv--;
}
if (q < n && q >= 0 && alive[q]) {
    if (a[j] > a[q]) inv--;
}

```

```

    }

    // perform merge
    a[i] = a[i] + a[j];
    // remove j
    alive[j] = false;
    nxt[i] = q;
    if (q < n && q >= 0) pre[q] = i;

    // update relations and inv count after merge
    if (p >= 0 && alive[p]) {
        if (a[p] > a[i]) inv++;
        push(a[p] + a[i], p);
    }
    if (q < n && q >= 0 && alive[q]) {
        if (a[i] > a[q]) inv++;
        push(a[i] + a[q], i);
    }

    ans++;
}

return ans;
}

```

Java:

```

class Solution {

    private static class Segment implements Comparable<Segment> {
        int index;
        long value;
        long mergeCost;
        Segment left;
        Segment right;

        Segment(int idx, long val) {
            index = idx;
            value = val;
        }

        @Override
        public int compareTo(Segment o) {
            if (right == null || o.right == null) {
                return right == null ? 1 : -1;
            }

```

```

    }
    long diff = mergeCost - o.mergeCost;
    if (diff != 0) return diff < 0 ? -1 : 1;
    return index - o.index;
}
}

```

```

public int minimumPairRemoval(int[] nums) {
    TreeSet<Segment> heap = new TreeSet<>();
    int violations = 0;

    Segment current = null;

    for (int i = 0; i < nums.length; i++) {
        Segment node = new Segment(i, nums[i]);

        if (current != null) {
            if (node.value < current.value) violations++;

            current.right = node;
            node.left = current;

            current.mergeCost = current.value + node.value;
            heap.add(current);
        }
        current = node;
    }

    heap.add(current);

    int operations = 0;

    while (violations > 0) {
        operations++;

        Segment best = heap.pollFirst();
        Segment next = best.right;

        if (next.value < best.value) violations--;

        best.value += next.value;
        best.mergeCost = best.value + (next.right != null ? next.right.value : 0);

        best.right = next.right;
    }
}

```

```

    if (next.right != null) {
        if (next.right.value < next.value) violations--;
        next.right.left = best;
        if (best.value > next.right.value) violations++;
    }

    heap.remove(next);
    heap.add(best);

    Segment prev = best.left;
    if (prev != null) {
        heap.remove(prev);

        if (prev.value > prev.mergeCost - prev.value) violations--;
        if (prev.value > best.value) violations++;

        prev.mergeCost = prev.value + best.value;
        prev.right = best;

        heap.add(prev);
    }
}

return operations;
}
}

```